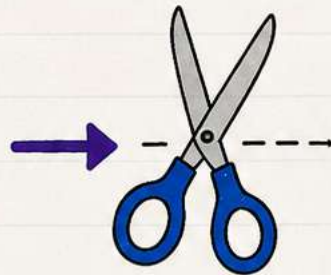
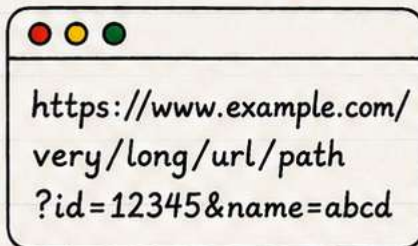


URL SHORTENER

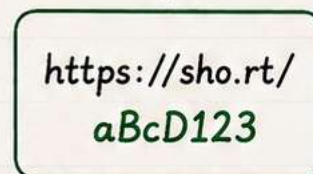
– SYSTEM DESIGN HANDBOOK –

Design a Scalable URL Shortener Service
like Bit.ly 😊

Long URL →



Short URL



Tiny link, Big Impact! ★

KEY NUMBERS (Approx)



100M+
URLs Created
(Lifetime)



10B+
Redirects
Per Day



< 50 ms
Redirect
Latency



Global
Users
Worldwide

WHY IS IT IMPORTANT ?

- ★ Used everywhere (sharing links)
- ★ Must be fast, reliable & highly available
- ★ Handles huge read traffic (redirects)
- ★ Simple for users, complex at scale! ❤️

TOPICS COVERED

- ✓ Requirements
- ✓ High Level Design
- ✓ URL Generation
- ✓ Database Design
- ✓ Caching
- ✓ Redirect Flow
- ✓ Scaling Strategy
- ✓ Analytics System
- ✓ Security
- ✓ Failure Handling
- ✓ Monitoring
- ✓ Trade-offs
- ✓ Best Practices



A short link is simple on the surface,
but the system behind it is a great example of scale, speed and smart design.



2. PROBLEM STATEMENT & REQUIREMENTS

Design a highly available and scalable URL Shortener service that converts long URLs into short, unique and easy to share links. 😊

2.1 Functional Requirements

- ① Shorten a long URL
- ② Redirect short URL to original long URL
- ③ Custom alias (user defined short code)
- ④ Set expiration time for links
- ⑤ Track & analyze clicks (analytics)
- ⑥ User authentication (optional)
- ⑦ Bulk URL shortening
- ⑧ QR code generation (optional)



2.2 Non-Functional Requirements

- **High Availability** : 99.99% uptime
- **Low Latency** : < 50 ms for redirect
- **High Scalability** : Handle billions of requests
- **Reliability** : No data loss
- **Consistency** : Short URL → Long URL mapping must be consistent
- **Security** : Prevent abuse, spam & malicious links
- **Durability** : Data should persist for long time



2.3 Daily Traffic Estimates (Approx)



10M+
Users
(Cumulative)



100M+
URLs Shortened
Per Day



10B+
Redirects
Per Day



< 50 ms
Avg Redirect
Latency



5 - 10 TB
Data Growth
Per Month

2.4 Key Constraints

- ★ Very large read to write ratio.
- ★ Short URLs must be unique.
- ★ Redirects should be extremely fast.
- ★ Analytics should not impact main redirect performance.
- ★ System should be highly available and fault tolerant.



2.5 Assumptions

- Each short URL is mapped to one long URL.
- Short codes are case sensitive.
- Users can register or use as anonymous.
- Analytics data can be processed asynchronously.
- We will support global users.

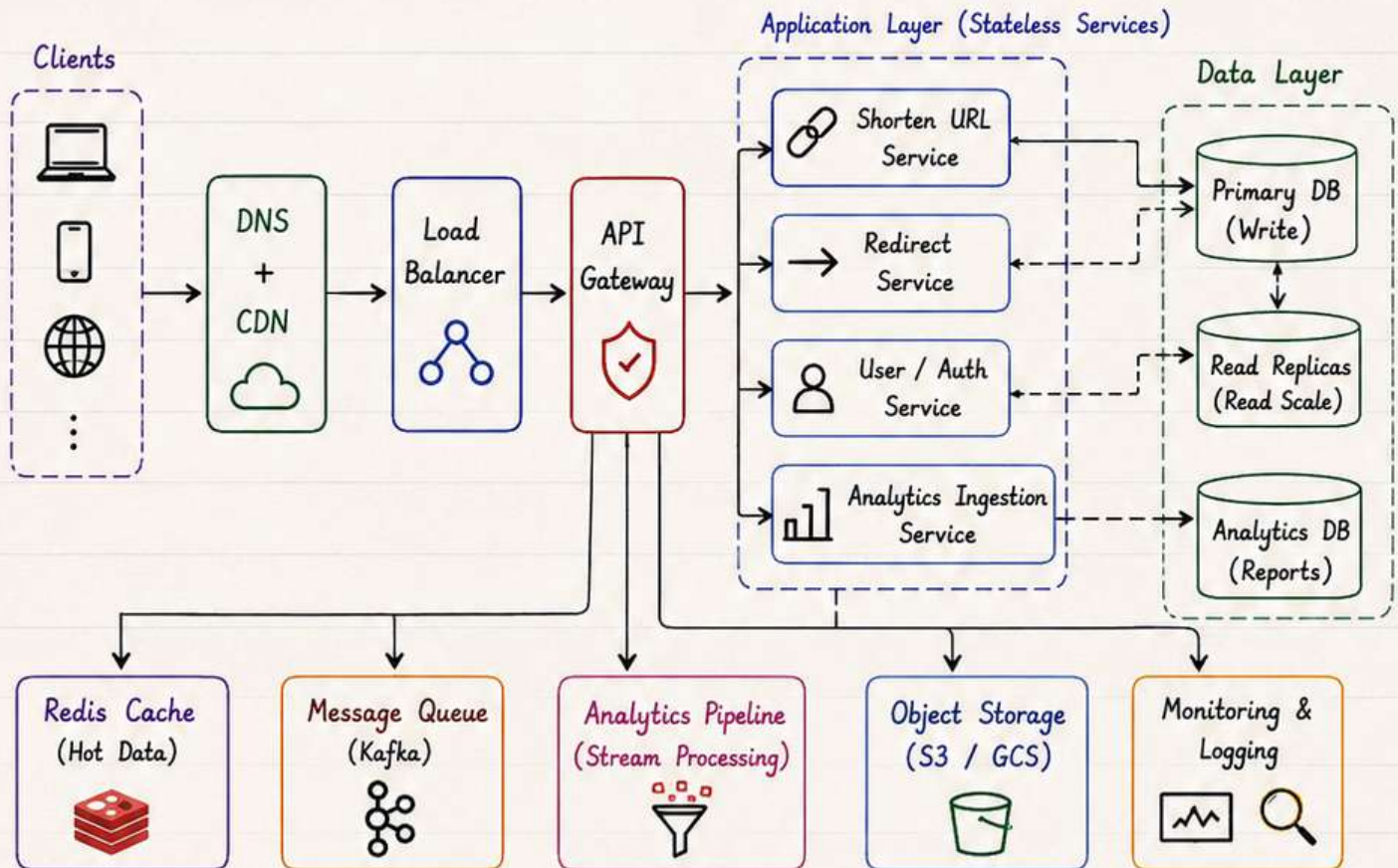


Our goal is simple - create short links that are **Fast, Reliable, Scalable** and **Easy** to share.



3. HIGH LEVEL ARCHITECTURE

We will design a highly available, scalable and low latency URL Shortener service that can handle billions of redirects. 😊



3.1 Components

- ① **Clients** → Web, Mobile Apps, Browsers
- ② **DNS + CDN** → Route users to nearest edge, cache static content
- ③ **Load Balancer** → Distribute traffic across multiple API servers
- ④ **API Gateway** → Single entry point, rate limiting, auth, routing
- ⑤ **Services** → Handle core business logic (stateless, horizontally scalable)
- ⑥ **Redis Cache** → Store hot URL mappings for fast redirects
- ⑦ **Databases** → Store URL data, users, analytics (with read replicas)

3.2 Data Flow (Shorten URL)

- ① User sends long URL to Shorten URL Service
- ② Service validates & generates short code
- ③ Check Redis (for uniqueness / custom alias)
- ④ If not exists → Write to Primary DB
- ⑤ Store mapping in Redis (TTL based)
- ⑥ Return short URL to user



Benefits of this Architecture

- ☆ Stateless services → easy horizontal scaling
- ☆ Cache + CDN → ultra low latency
- ☆ Read Replicas → scale read heavy traffic
- ☆ Async analytics → no impact on core flow
- ☆ Highly Available & Fault Tolerant



Keep the core path (shorten + redirect) fast & simple.
Move heavy work (analytics, reports) to async pipelines.



4. SHORT URL GENERATION

Goal : Generate short codes that are unique, hard to guess, compact and URL-safe. 😊

4.1 Design Choices

Approach	How it Works	Pros	Cons	Used By
1. Auto Increment + Base62	Use an auto increment ID and encode it in Base62	✓ Fast, Simple ✓ Short codes	✗ Predictable IDs ✗ Need encoding	bit.ly, TinyURL (initially)
2. Hashing (MD5 / SHA)	Hash the long URL and encode part of hash	✓ Deterministic ✓ No DB write (on first use)	✗ Longer codes ✗ Collision possible	Some early implementations
3. Random String (Base62)	Generate random string from Base62 characters	✓ Hard to guess ✓ Very unique	✗ Slightly larger db/check needed	Many systems
4. UUID (Base62 Encoded)	Use UUID and encode in Base62	✓ Globally unique ✓ No collision	✗ Longer length	Internal tools
5. Snowflake ID + Base62	Use timestamp based Snowflake ID → Base62	✓ Unique, sortable ✓ High throughput	✗ Slightly complex ✗ Needs infra	Large scale systems

4.2 Base62 Encoding

Characters Used (62)

0-9 (10)
a-z (26)
A-Z (26)

Example Encoding

ID (Decimal) : 123456

$123456 \div 62 = 1991 \text{ rem } 14 = e$
 $1991 \div 62 = 32 \text{ rem } 7 = 7$
 $32 \div 62 = 0 \text{ rem } 32 = G$

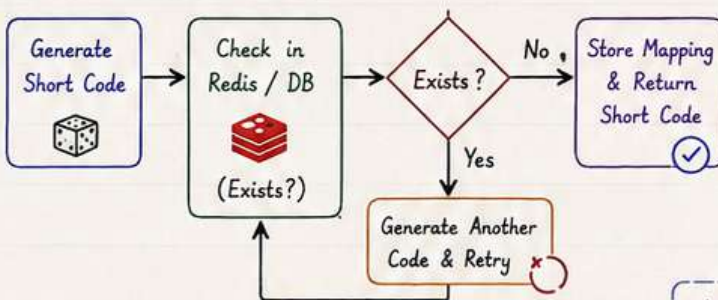
Read Bottom to Top

Short Code : **G7e**

4.3 Base62 Table (Index → Character)

Index	0	1	2	3	4	5	6	7	8	9	10	...	35	...	61
Char	0	1	2	3	4	5	6	7	8	9	a	...	z	...	Z

4.4 Collision Handling



4.6 Code Length

Length	Total Combinations (Approx)
5 chars	$62^5 \approx 916 \text{ Million}$
6 chars	$62^6 \approx 56.8 \text{ Billion}$
7 chars	$62^7 \approx 3.5 \text{ Trillion}$
8 chars	$62^8 \approx 218 \text{ Trillion}$

★ 6-7 characters are ideal for most systems (balance between short & unique).

4.5 ID Strategy (Recommended)

Use Snowflake ID → Base62 Encoding

41 bits	10 bits	12 bits
Timestamp (millis)	Worker ID	Sequence

✓ High throughput ✓ Unique ✓ Time sortable

Then encode the 64-bit ID in Base62.

💡 This gives short, unique and scalable codes.

4.7 Best Practices

- ✓ Use Base62 to keep URLs short & URL-safe
- ✓ Avoid sequential/predictable codes
- ✓ Always check uniqueness before storing
- ✓ Store mapping in cache (Redis) for fast lookup
- ✓ Keep code length consistent (6-7 chars)
- ✓ Support custom aliases with uniqueness check
- ✓ Rate limit creation to prevent abuse



A good short code is short, unique, hard to guess and fast to generate.

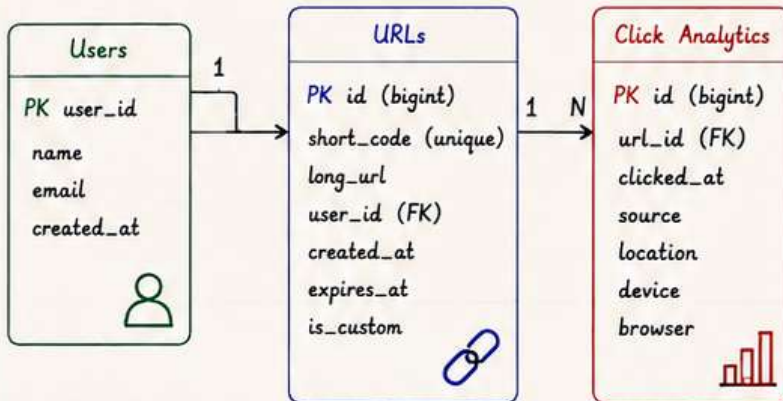
Base62 + Smart ID strategy = Scalable URL Shortener.



5. DATABASE DESIGN

We need a robust data model to store mappings, users and analytics efficiently and support high read & write scale. 😊

5.1 ER DIAGRAM (Simplified)



PK - Primary Key FK - Foreign Key 1:N - One to Many

5.2 URLs Table (Fields)

Field	Type	Description
id	BIGINT	Auto increment ID
short_code	VARCHAR(10)	Unique short code
long_url	TEXT	Original long URL
user_id	BIGINT	Owner of the link
created_at	DATETIME	When link created
expires_at	DATETIME	Expiry time (optional)
is_custom	BOOLEAN	Custom alias or not
click_count	BIGINT	Total clicks (cached)

Interview Tip

Use **BIGINT** for ID & counters.

Keep **short_code** indexed (**UNIQUE**) for fast lookup. 😊

5.3 Indexing Strategy

- ★ PRIMARY KEY on id
- ★ UNIQUE INDEX on short_code
- ★ INDEX on user_id (for user links)
- ★ INDEX on created_at (analytics queries)
- ★ PARTITION Click Analytics by date (daily/monthly)



Interview Tip

Short redirects are read heavy.
Optimize for fast lookups using short_code index. 😊

5.4 SQL Example

Insert new URL mapping

```

INSERT INTO urls (short_code, long_url, user_id,
created_at, expires_at, is_custom)
VALUES ('aBcD123', 'https://www.example.com/very/long/url',
101, NOW(), NULL, false);
  
```

SQL

Fetch original URL for redirect

```

SELECT long_url FROM urls
WHERE short_code = 'aBcD123' AND expires_at IS NULL;
  
```

SQL

5.5 SQL vs NoSQL

SQL (MySQL / PostgreSQL)

- ✓ Strong consistency
- ✓ ACID transactions
- ✓ Great for relational data & complex queries
- ✓ Better for small to medium scale
- ✓ Easy joins (Users, URLs, Analytics)

NoSQL (Cassandra / MongoDB)

- ✓ Horizontal scaling
- ✓ High write throughput
- ✓ Flexible schema
- ✓ Better for huge scale analytics
- ✓ Eventual consistency (base design)

Interview Tip

Use SQL for core mapping (strong consistency).
Use NoSQL / Columnar DB for analytics (scale). 😊

5.6 Data Growth Estimation (Approx)



100M+ new URLs per month
~ 3.3M per day



Click events ~ 10B per day



Mapping data ~ 5-10 TB per month



Global distribution needed

Design Insight

Keep writes fast (async analytics).

Reads must be super fast (< 50 ms). 😊

5.7 Best Practices



Keep URLs unique & secure



Use indexed short_codes



Cache hot mappings in Redis



Partition analytics by date



Use Read Replicas for scale



Regular backups & monitoring



A well designed database is the backbone of a URL Shortener.

Fast lookups + Scalable storage = Great user experience 😊

6. REDIRECT FLOW

Let's see how a redirect happens when a user clicks on a short URL. 😊

① User Clicks Short URL

User enters `https://sho.rt/abc123` in browser or clicks the link.

② Request Hits Edge / CDN

DNS resolves the domain and the request goes to CDN / Edge server (Cloudflare, etc.)

③ Forward to Load Balancer

CDN forwards the request to Load Balancer.

④ URL Service Lookup

Request is routed to URL Service to fetch the original long URL using the short code.

⑤ Cache Lookup (Redis)

Check if short code exists in Redis.
If yes → return the long URL.
Else → go to Database.

⑥ Database Lookup (On Miss)

Query Primary DB (or Read Replica) to get the long URL mapping.
Store it back in Redis with TTL.

⑦ Return Redirect Response

Service returns 301/302 redirect response to the user with the original long URL.

⑧ User Redirected

Browser redirects the user to the original long URL (`https://example.com/very/long/url`).

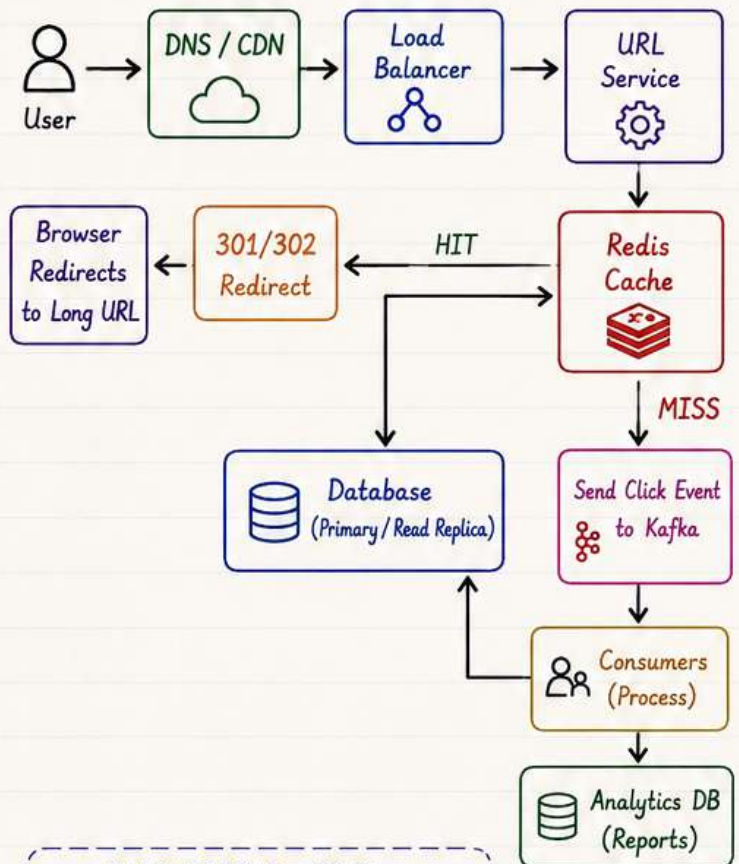
⑨ Analytics Event Generated

Click event is sent to Kafka for async processing.

⑩ Analytics Processing (Async)

Consumers process the event, update counters, and store analytics.

END-TO-END FLOW DIAGRAM



HTTP STATUS CODES

301 → Permanent Redirect (Preferred)

302 → Temporary Redirect

💡 Interview Tip

Why use Cache first?

- ★ Redirects are read heavy (~1000:1)
- ★ Cache gives sub-millisecond lookups
- ★ DB is slow & expensive
- ★ Always use TTL to keep cache fresh

🔗 Common Follow-up Questions

1. What if Cache & DB both are down?
2. How will you handle expired links?
3. How will you track unique clicks?
4. How will you prevent bots / spam?
5. How will you handle very hot URLs?

KEY TAKEAWAY

Keep the read path fast → Cache first, DB next.
Always process analytics asynchronously.

7. CACHING & PERFORMANCE

Caching is critical for ultra-low latency redirects and reducing DB load.

Let's see how we design it. 😊

7.1 Why Cache?

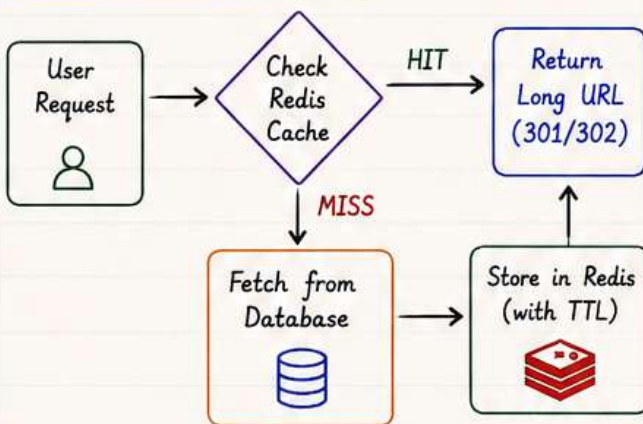
- ★ DB lookups are expensive.
- ★ Most URLs are read-heavy.
- ★ Caching reduces latency to sub-millisecond.
- ★ Protects DB during traffic spikes.
- ★ Improves overall system throughput.



Real-World Example 🌐

Bitly uses multiple layers of cache (Redis + CDN) to serve 99%+ redirects without hitting the database. Only cache misses go to DB. This helps handle billions of redirects/day. 😊

7.2 Cache Architecture



7.3 Cache Aside Pattern (Used)

- ① Application checks cache.
- ② If HIT → return data.
- ③ If MISS → fetch from DB.
- ④ Store result in cache with TTL.
- ⑤ Return data to user.

Interview Tip 💡

- ★ Cache-Aside is the most commonly used pattern in real systems.
- ★ Write-Through is rarely needed here since writes are infrequent.

7.4 TTL Strategy

Type	TTL (Approx)
Normal URLs	24 Hours
Hot URLs	7 Days
Very Hot URLs	30 Days
Custom Aliases	1-7 Days
Expired Links	5-15 Minutes (Negative Cache)

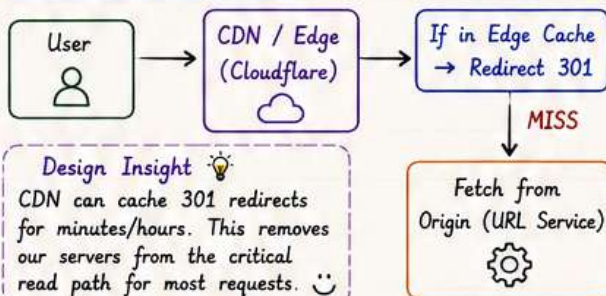
⌚ Long TTL for hot data, short TTL for changes.

7.5 Hot Keys Problem

- ★ Some short codes are accessed millions of times.
- ★ Even Redis can become a bottleneck.
- ★ Solutions:
 - Increase TTL for hot keys
 - Use Local In-Memory Cache (Caffeine / Guava)
 - Use CDN for redirects (Edge caching)
 - Replicate Redis (Cluster Mode)



7.6 CDN for Redirects

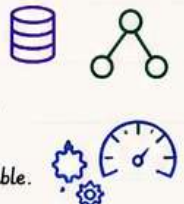


Design Insight 💡

CDN can cache 301 redirects for minutes/hours. This removes our servers from the critical read path for most requests. 😊

7.7 Other Performance Optimizations

- ✓ Use connection pooling to DB.
- ✓ Use Read Replicas for read scaling.
- ✓ Use Async I/O (Non-blocking) in services.
- ✓ Compress responses (gzip).
- ✓ Keep service stateless & horizontally scalable.
- ✓ Batch analytics events (async).



7.8 Cache Metrics to Monitor



- Cache Hit Rate (Should be > 90%)
- Cache Latency (p50, p99)
- Eviction Rate
- Memory Usage
- Hot Keys / Top URLs

Interview Question ?

What happens if cache & DB both are down?

Answer: Return graceful error (503) and serve a cached friendly error page from CDN if possible. 😊

KEY TAKEAWAY

Good caching = Happy users + Happy DB.
Cache first, DB next, CDN everywhere! 😊



8. RATE LIMITING & SECURITY

Rate limiting and security protect the service from abuse, DDoS attacks and scraping. 😊

8.1 Why Rate Limit?

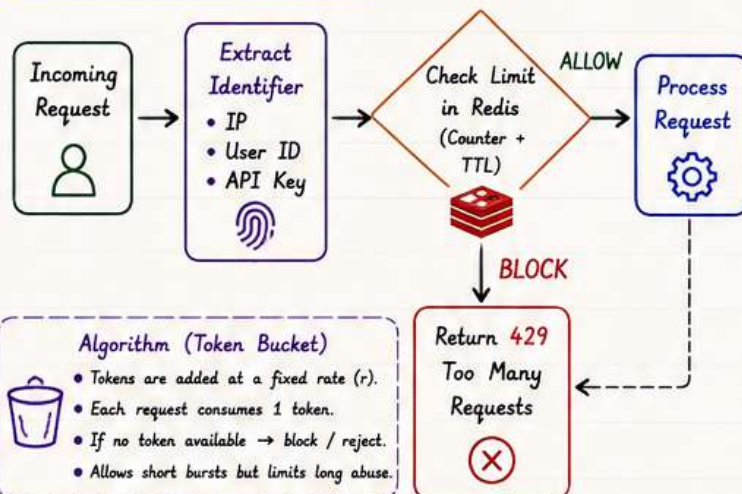
- ☆ Prevent abuse (spam, brute force, scraping).
- ☆ Protect backend & database from overload.
- ☆ Ensure fair usage for all users.
- ☆ Avoid high infra cost.



Real-World Example 🌐

Bitly limits anonymous users to ~100 requests/min per IP and authenticated users get higher limits. This stops bots from creating millions of short URLs and keeps the service stable. 😊

8.2 Rate Limiting Flow



8.3 Rate Limit Key Strategy

Key Type	Use Case
IP Address	For anonymous users
User ID	For authenticated users
API Key	For third-party integrations
User + IP	For extra strict control

Interview Tip 💡

- ☆ Choose key based on your system's abuse pattern.
- ☆ Usually IP for anon users and UserID for logged-in users.

8.4 Rate Limiting in Redis

```

INCR key      # increment counter
EXPIRE key 60 # set TTL (window)
GET key       # get current count
  
```

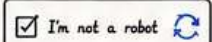
If count > limit → block request

Use Lua script to make INCR + EXPIRE atomic.



8.5 Other Security Measures

- ✓ Validate and sanitize all inputs.
- ✓ Use HTTPS everywhere.
- ✓ Protect against SQL Injection / XSS.
- ✓ Use CAPTCHA for suspicious activity.
- ✓ Use WAF (Web Application Firewall).
- ✓ Encrypt sensitive data at rest and in transit.



8.6 Rate Limit Example

Limit: 100 requests / minute per IP

User IP: 1.1.1.1 makes 120 requests in 1 minute

→ First 100 requests → **ALLOWED**

→ Next 20 requests → **BLOCKED (429)**



Common Mistakes ⚠

- ✗ Using only IP for logged-in users.
- ✗ Not setting TTL → counter never resets.
- ✗ Rate limiting after heavy processing.
- ✗ Storing counters in DB (too slow!).
- ✗ Not handling clock skew in distributed systems.

Interview Questions ?

1. How will you design rate limiting for this system?
2. Where will you apply rate limiting (CDN, API Gateway, or Service)?
3. How will you store counters and handle scalability?
4. How will you prevent brute force and URL guessing?
5. What is Token Bucket vs Leaky Bucket?



KEY TAKEAWAY

Rate limiting keeps the system safe and fair.

Redis + Smart Keys + Proper Limits = Secure & Scalable URL Shortener



9. DATA CONSISTENCY & AVAILABILITY

We must ensure correct, consistent data while keeping the system highly available and low latency. 😊

9.1 Consistency Models

- ★ Strong Consistency : All nodes see the same data at the same time.
- ★ Eventual Consistency : Updates propagate asynchronously, system converges over time.
- ★ Session Consistency : Reads reflect user's own writes.
- ★ Causal Consistency : Respect the cause-and-effect order.

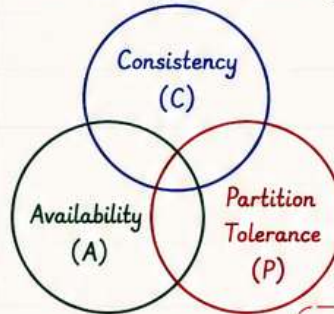


Real-World Example 🌐

Social media timelines use eventual consistency, A like may take a few seconds to appear, but the system stays fast and highly available. 😊

9.2 CAP Theorem

In a distributed system, you can have only TWO out of the THREE guarantees:



Remember 💡

- ★ CA : RDBMS (Not partition tolerant)
- ★ CP : HBase, MongoDB
- ★ AP : Cassandra, DynamoDB

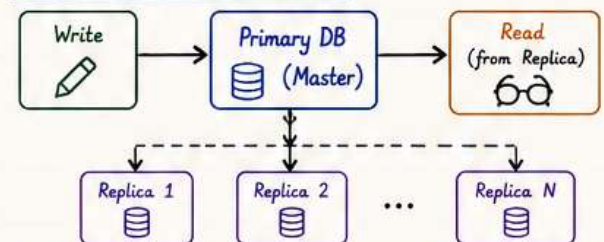
Interview Tip 💡

For most internet-scale systems, we choose AP (Availability + Partition Tolerance) and design for eventual consistency. 😊

9.3 Database Choice

DB Type	Examples	Pros	Cons	Use Case
Relational 	MySQL, Postgres	Strong consistency, ACID, Complex queries	Vertical scaling limits, harder to shard	User data, payments, transactions
NoSQL (Document) { }	MongoDB	Flexible schema, Horizontal scale, High availability	Eventual consistency (by default)	User profiles, catalogs, content
NoSQL (Key-Value) 	Redis, DynamoDB	Super fast, Simple, Great for cache	Limited query capabilities	Caching, sessions, rate limiting, counters

9.4 Read/Write Strategy



Interview Tip 💡

Writes go to Primary to ensure consistency. Reads go to Replicas for better performance and scalability. 😊

9.5 Handling Inconsistencies

- ★ Use versioning / timestamps.
- ★ Use idempotent writes to avoid duplicates.
- ★ Use conflict resolution (Last Write Wins, Merge, or Custom Logic).
- ★ Background reconciliation jobs for eventual consistency.



9.6 Idempotency (Important!)

An operation can be called multiple times but has the same effect as calling it once.

How?

- Use idempotency keys (UUID).
- Store the key + response.
- If same key comes again → return stored response.



9.7 Tools & Patterns



Redis

Cache, Counters, Rate Limiting



Kafka

Event Streaming, Async Processing



Postgres

Strong Consistency, ACID Transactions



MongoDB

Flexible Schema, High Availability



Read Replicas

Scale Reads, Improve Latency



Idempotency Keys

Safe Retries, No Duplicates

⊗ Common Mistakes

- ✗ Choosing wrong consistency model.
- ✗ Not handling partial failures.
- ✗ Ignoring replication lag.
- ✗ Forgetting idempotency for writes. 😊

⚖️ Trade-off

Strong Consistency vs High Availability
Pick what matters more for your use case.
There is no one-size-fits-all! 😊

? Interview Question

How will you design a system that remains available even if one datacenter goes down?
Hint: Replication, Multi-AZ, Failover, Eventual Consistency. 😊

KEY TAKEAWAY

Balance consistency, availability and partition tolerance.

Use the right DB, caching, replication and patterns to build reliable systems. 😊



10. MONITORING, LOGGING & ALERTS

You can't improve what you can't measure.

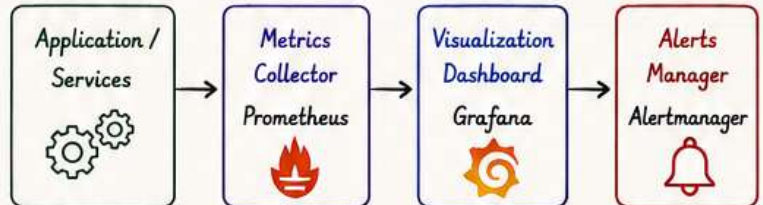
Observability helps us detect issues early and keep the system healthy. 😊

10.1 What to Monitor?

- ☆ Request Rate (RPS)
- ☆ Error Rate (4xx, 5xx)
- ☆ Latency (p50, p95, p99)
- ☆ Cache Hit / Miss Ratio
- ☆ Database QPS & Slow Queries
- ☆ CPU, Memory, Disk, Network
- ☆ Active Users / Sessions
- ☆ Queue Lag (Kafka)
- ☆ System Availability (Uptime)



10.2 Observability Stack (Example)



Interview Tip 💡

- ☆ Monitor RED Metrics: Rate, Errors, Duration.
- ☆ Set alerts on SLOs, not just infra metrics.
- ☆ Dashboards should answer: "Is my system healthy?" 😊

10.3 Logging Strategy

- ✓ Structured logs (JSON) for easy parsing.
- ✓ Include: timestamp, level, request_id, user_id, service, message, metadata.
- ✓ Centralize logs for search & analysis.
- ✓ Set log levels: DEBUG, INFO, WARN, ERROR.
- ✓ Retain logs for required period (e.g., 7-30 days).



Real-World Example 🌐

During a traffic spike, error rate increased. Logs helped us find a DB connection leak in a specific service within minutes and fixed it. 😊

10.4 Alerting Best Practices

- ✓ Alert on symptoms, not on causes.
- ✓ Avoid alert fatigue (too many false alerts).
- ✓ Use meaningful thresholds (baseline + deviation).
- ✓ Group related alerts.
- ✓ Have escalation policy (on-call rotation).
- ✓ Test your alerts regularly.



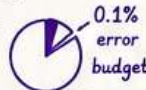
Common Mistakes ☹️

- ✗ Alerting on 100% CPU always.
- ✗ Noisy alerts during deployments.
- ✗ No runbook / steps for alerts.
- ✗ Ignoring small errors for long time.



10.5 SLO, SLA & Error Budget

- SLO (Service Level Objective): Target for availability/latency (e.g., 99.9% uptime)
- SLA (Service Level Agreement): Promise made to users / customers.
- Error Budget: The amount of unreliability we can tolerate. e.g., 99.9% uptime → 0.1% error budget (~43 minutes / month)

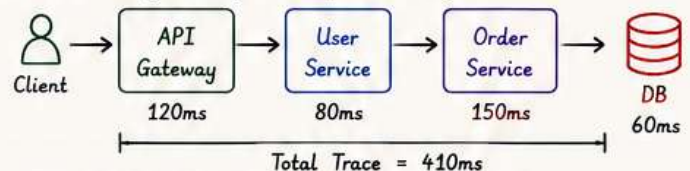


Interview Tip 💡

Use error budget to decide when to release or roll back a risky change. 😊

10.6 Tracing (Distributed Systems)

- ✓ Each request gets a trace_id.
- ✓ Trace is propagated across all services.
- ✓ Helps to find latency bottlenecks.
- ✓ Tools: Jaeger / Zipkin / OpenTelemetry.



Design Insight 💡

Tracing shows where time is spent, not just that it is slow. 😊

10.7 Monitoring & Alerting Flow



KEY TAKEAWAY

Monitor what matters, alert smartly, and act quickly.

Observability = Reliability + Happy Users 😊

12. SCALABILITY, TRADE-OFFS & CONCLUSION

Scaling is never free. We balance cost, complexity and performance.

Great design is all about smart trade-offs. 😊

12.1 How We Scale This System

- ☆ Horizontal scaling of all stateless services.
- ☆ Auto-scaling based on CPU / RPS.
- ☆ Sharding / Partitioning in Database.
- ☆ Read replicas for heavy read workloads.
- ☆ Edge caching (CDN + Redis) for low latency.
- ☆ Async processing for analytics & heavy tasks.
- ☆ Background jobs for reconciliation & cleanup.
- ☆ Multi-AZ deployment for high availability.



12.2 Key Trade-offs

Trade-off	Option A	Option B	What to Choose?
Cache	More Cache (Redis)	Less Cache	More cache for low latency reads
Consistency	Strong	Eventual	Eventual for scalability
Database	Single DB	Sharded DB	Shard when data grows
Write Path	Sync Write	Async Write	Async for high throughput
Analytics	Real-time	Batch (Async)	Batch is cheaper & scalable



Interview Tip

Interviewers love trade-off based questions.

Justify your choice with scale, cost & user impact. 😊

12.3 Capacity Estimation (Rough)



MAU (Monthly Active Users) : 10 Million
Avg Clicks / User / Day : 10
Total Clicks / Day : 100 Million
Peak QPS (10x) : ~12,000 QPS



Assume short code length = 6 chars
Possible codes = $62^6 \approx 56.8$ Billion
Plenty of room for growth!



Target Latency:
P95 < 50ms | P99 < 100ms



Cost Optimization:
Use caching, CDN & async processing.
Pay for what you use (serverless / autoscale).

12.4 Bottlenecks & How We Handle



High Read Load → Redis Cache + CDN



DB Overload → Read Replicas + Sharding



ID Generation → Snowflake IDs (Distributed)



Analytics Load → Kafka + Async Processing



Single Point Failure → Multi-AZ + Health Checks

Real-World Example 🌐

bit.ly started with a single server.

As traffic grew, they added caching, CDNs, and sharded storage to handle billions of clicks. 😊

12.5 Cost Drivers

- Compute (EC2 / Containers / Lambdas)
- Database (RDS / Read Replicas / Storage)
- Cache (Redis Cluster)
- Network (CDN, Data Transfer)
- Logging, Monitoring & Alerting
- Third-party services (SMS, Email, etc.)



Optimize before you scale!

12.6 Monitoring Metrics to Track



Request Rate (RPS)



Error Rate (4xx/5xx)



Latency (P50, P95, P99)



DB Connections



Cache Hit Ratio



System Uptime



Interview Tip

Always monitor RED (Rate, Errors, Duration) +

Golden Signals (Latency, Traffic, Errors, Saturation). 😊

Common Mistakes ☒

- ✗ Storing long URL in DB instead of cache.
- ✗ No rate limiting → system can be abused.
- ✗ Ignoring hot keys & cache TTL.
- ✗ No monitoring & alerting.
- ✗ Not thinking about scale & availability early. 😞

Interview Questions ?

1. How will you scale this to 1 Billion requests/day?
2. What if Redis goes down?
3. How will you handle a viral URL?
4. How will you ensure data consistency?
5. What are the biggest bottlenecks in this design? 😊



Must Remember ★

- ☆ Cache first, DB next.
- ☆ Keep services stateless.
- ☆ Monitor everything.
- ☆ Design for failure.
- ☆ Think scale from Day 1! 😊



KEY TAKEAWAY

A tiny URL hides big system design concepts.

Good systems are simple, scalable, observable and user-friendly. 😊

13. FINAL CHEAT SHEET (2-3 MIN REVISION)

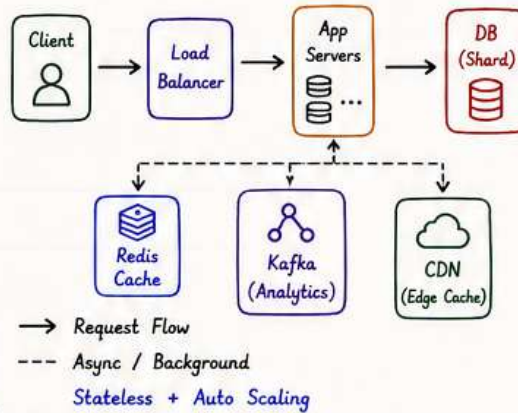
System Design Interview — Quick Recall ☺

1. CORE REQUIREMENTS

- ★ Shorten long URLs
- ★ Unique & non-guessable IDs
- ★ High availability (99.9%+)
- ★ Low latency (< 50ms)
- ★ High read & write throughput
- ★ Scalable (Millions / day)
- ★ Analytics & monitoring
- ★ Secure & abuse protected



2. HIGH LEVEL ARCHITECTURE



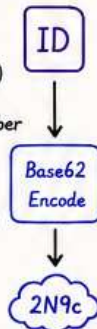
3. URL SHORTENING FLOW

- ① User sends a long URL
- ② App generates Short ID
- ③ Check Redis (exists?)
 - HIT → Return short URL
 - MISS → Write to DB + Cache
- ④ Return short URL to user
- ⑤ On access, redirect to long URL



4. ID GENERATION

- ▶ Use Base62 (0-9, a-z, A-Z)
- ▶ Encode auto-increment number (e.g., 123456 → 2N9c)
- ▶ Snowflake / ULID / KSUID
- ▶ Ensure non-sequential & unique



5. DATABASE DESIGN (Simplified)

Table	Purpose
urls	id (PK), short_code, long_url, user_id, created_at, clicks, expire_at
users	id, email, password (hashed), plan, created_at
clicks	id, url_id, timestamp, ip, user_agent, country, referrer
url_stats_daily	url_id, date, clicks, unique_clicks
blacklist	id, short_code, reason, created_at

6. CACHING (REDIS)



- ✓ Cache key : short_code → long_url
- ✓ TTL : e.g., 24 hours
- ✓ Warm cache for hot URLs
- ✓ Use Redis Cluster for scale
- ✓ Evict using LRU policy

7. RATE LIMITING & SECURITY

- ✓ IP based rate limiting (e.g., 100 req/min)
- ✓ User based rate limiting (stricter)
- ✓ Use Redis (INCR + EXPIRE)
- ✓ Protect endpoints (WAF, CAPTCHA)
- ✓ Validate & sanitize all inputs
- ✓ Use HTTPS everywhere



8. DATA CONSISTENCY

- ✓ Strong Consistency : Same data everywhere (slower)
- ✓ Eventual Consistency : Updates propagate (faster & scalable)
- ✓ Use Read Replicas for scaling
- ✓ Background reconciliation jobs



9. SCALABILITY STRATEGIES

- ✓ Horizontal scaling (stateless services)
- ✓ DB Sharding (by short_code / user_id)
- ✓ Read Replicas for heavy reads
- ✓ CDN for redirects (edge caching)
- ✓ Async processing (Kafka, Queues)
- ✓ Multi-AZ / Multi-Region deployment



10. TRADE-OFFS



- Cache → Speed ↑ but extra complexity
- Consistency → Strong vs Eventual
- DB → Single (simple) vs Sharded (scale)
- Write Path → Sync (safe) vs Async (fast)
- Analytics → Real-time (costly) vs Batch (cheap)
- ID Gen → Simple (auto-inc) vs Snowflake (scale)

11. COMMON MISTAKES



- ✗ Storing logs in primary DB
- ✗ No rate limiting → abuse
- ✗ Ignoring cache TTL
- ✗ No monitoring or alerting
- ✗ Not thinking about scale early
- ✗ Hot keys & DB hotspots
- ✗ No backups & failover plan



12. REAL-WORLD EXAMPLE



bit.ly handles billions of requests/day using:

- Auto-scaling, CDN, Redis
- Sharded storage
- Async analytics pipeline
- Multi-AZ for HA

Result: Fast, reliable & highly scalable! ☺

13. INTERVIEW TIPS



- ★ Clarify requirements first.
- ★ Break problem into smaller components.
- ★ Explain trade-offs clearly.
- ★ Communicate assumptions.
- ★ Draw diagrams.
- ★ Discuss scale, reliability & cost.



14. SAMPLE INTERVIEW QUESTIONS



- ① How will you generate short IDs?
- ② How will you handle 1 Billion req/day?
- ③ How will you prevent abuse?
- ④ How will you store & serve data?
- ⑤ What are the biggest bottlenecks?
- ⑥ How will you ensure high availability?



15. MUST REMEMBER



- ★ Cache first, DB next.
- ★ Keep services stateless.
- ★ Design for failure.
- ★ Monitor everything.
- ★ Automate scaling.
- ★ Think scale from Day 1!



KEY TAKEAWAY



A good system is FAST, RELIABLE, SCALABLE & COST EFFECTIVE.

Understand trade-offs, design smart & keep users happy!

